

Marv Predict Max - Handoff Brief

Prepared 2026-10-09 for the next assistant taking over. Owner: moonrider1968@gmail.com. Everything below is what has been built, what was measured, what is deployed, and what is still open. Numbers are walk-forward backtests on seasons the models did not train on unless stated.

1. The one-paragraph truth

- **No proven betting edge has been found.** The model projects games well enough to pick heavy favorites (high win rate), but the closing line beats it, so profit after the vig is about break-even where measured.
- High accuracy comes from backing heavy favorites (-400 to -1000). Never present targets (for example '90%+ guaranteed' or the 100% figures in the owner's PDFs) as results.
- Everything is **paper-tracked** (PAPER_MODE on) and graded in ledgers. Stakes shown on cards are 0.5u leans. Do not turn any signal into real-money picks until it has 100+ graded bets above 52.4% (break-even at -110).

2. What the system is

A multi-sport Telegram prediction bot running on the owner's Google Compute Engine VM (code in /opt/marv-bot, runs as user marvbot). Repo: github.com/marvy1968/marvy1968, working branch **claude/analysis-ak180w**. Read README.md, ANALYSIS.md and CLAUDE.md first.

Active sports: NFL, college football top 30 (cfb), EuroLeague, WNBA, NBA (preseason skipped), NCAA men's (ncaab) and women's (ncaaw) basketball. MLB/NHL/soccer exist but are off.

Pipeline (python -m marv run): load schedule + odds -> stats experts project each team's score (ridge 'stat formula', random forest, opponent-adjusted power ratings; consensus) -> shared Monte Carlo simulator -> veto stack (expert agreement, backtested confidence floors, key injuries, stale stats, backup QB, wind, road favorites) -> Telegram card + pick ledger + SQLite audit + PDF chart.

3. Services on the VM (systemd)

Unit	What it does
marv-bot.timer / marv-bot-late.timer	Daily run 10:00 ET, late run 17:30 ET (+11:45 Sun). Sends sport cards and the daily bet card.
marv-live.service	In-game alerts after each quarter/half, only for NEW edges of 6%+ vs the live price; wins announced at the final. Also records NFL live-prop snapshots (data collection only).
marv-edges.service	Edge board + Telegram query bot (/board, /game, /gaps, /check, /grade). Alerts new edges and Bovada-vs-Pinnacle gaps.
marv-bridge.service	Local fair-price API on 127.0.0.1:8787 for the owner's March_edge bot.
marv-health.timer	Every 30 min: Telegram alert only if a service is down, the daily run failed/missed in 26h, disk 90%+, or Odds API credits under 2,000.
marv-vmmlink.timer	Every minute: runs allowlisted jobs that an assistant pushes to the vm-link branch (see section 8).

Alert rules: unsolicited Telegram messages only cover games starting today (Eastern); queries (/game, /board, /gaps) cover everything. Alerts go to the owner's chat and are mirrored to the second bot (@March_betbot). Messages are prefixed with the MARV label so they are not confused with March_edge.

4. Backtest results (keep claims honest)

Sport	Test seasons	All games	Experts agree	80%+ conf.	85%+ conf.
NCAA men's	2023-25	73.9%	78.0%	90.8% (~2,300/yr)	93.9% (~1,700/yr)
NCAA women's	2023-25	77.4%	81.9%	91.5% (~2,700/yr)	94.2% (~2,150/yr)
WNBA	2023-25	66.6%	70.0%	84.1% (~38/yr)	87.2% (~16/yr)
NBA	2024-26	66.8%	70.0%	84.0% (~225/yr)	87.2% (~104/yr)
NFL	2023-25	64.8%	68.1%	79.5% (~26/yr)	95.5% (22 picks, tiny sample)
College football	2023-25	77.0%	78.8%	88.3% (~134/yr)	91.9% (~102/yr)

- College basketball rows include lopsided games against lower-division teams. The earlier D1-vs-D1-only figures were about 90.3% (men's) and 90.8% (women's); use those.
- ROI after vig is about break-even wherever lines exist (NFL held-out 2024-25: spread +1.3%, total +1.5%, moneyline +3.0% on ~80 bets each = noise).
- Over/unders: NFL and college football about 50% (no edge); NBA 52.8% at a 17% edge threshold (break-even). Weather and rest did not help totals.
- Brier score NFL: market 0.2059 vs model 0.2205. The closing line is more accurate than the model.

5. Experiments run and what they showed

Idea	Result
Weighted head-to-head stat tally (ridge weights, opponent-adjusted, 75/25 season + last-3 blend)	Small gain over equal weights; opponent adjustment and 75/25 blend helped modestly. No ROI edge.
Recency weighting (last 25%, exponential decay)	No reliable improvement over the existing blend.
Team over/under trends, continuation vs reversal	College football trend FADES backtested about 55-57% (paper only). Tags: OVER-FADE, OVER-FADE+MOVE, OVER-FADE+INFLATED, UNDER-FADE; NFL TOTAL-INFLATED under 55.8%. Needs 100+ graded bets before real use.
Line movement / Bovada vs Pinnacle	College totals where Bovada is 0.5+ off Pinnacle: 54.4% over 2,340 games, +4.6% ROI (2014-19). The one lead with a real-price backtest. NFL untested.
Monte Carlo football simulator calibration (key numbers, garbage time, endgame)	Did not improve picks. Reported honestly.
NFL player props vs real Bovado/DK/FD lines (2025)	49% over 3,040 bets, ROI -5%. No edge. Props are OFF by default (PROPS=on to enable).
Injury trend ('regimes'): backup QB or WR1 out -> scale projection by scoring in the new situation (shrunk $n/(n+2)$, cap 25%)	Built and live; not yet backtested. Backup-QB (under 3 starts) vetoes all bets on the game.
Model vs market stack (this week)	Model adds ~0 beyond the closing spread (NFL coefficient 0.00-0.01 vs market 0.14; same for college football). Totals: model adds nothing (top-20% edge hit 50.2%/50.4%).

6. Latest engine change: market-anchored confidence

- Stats-mode moneyline confidence for **NFL and college football** is now $0.2 \times \text{model} + 0.8 \times \text{no-vig market}$ (StatsRules.ml_model_weight in marv/stats/live.py; set in marv/stats/registry.py). Floors 0.85.

- At equal pick volume this beats model-only: NFL top 400 picks 84.8% vs 76.5%; at 85%+ confidence NFL 88.6-90.5%, college football 93.4% (900 games) vs 91.5% model-only.
- This is the market's information, not new skill. ROI stays about break-even. Not yet tested for college basketball or WNBA (needs the VM's historical lines).

7. Daily bet card and paper signals

- `marv/betcard.py`: one card per day with the best 2-10 bets across all sports. Tiers: HIGH (passed every veto) > SIGNAL (backtested paper signals) > MODEL (edge 3%+, soft vetoes only). Max 2 per game; games with a backup QB, key injury or stale data are excluded. Logged to `state/betcard_log.json` and graded daily, with closing-line value (CLV) tracking.
- Paper-tracked ledgers: `state/picks.json`, `ou_tags_log.json`, `sharp_gap_log.json`, `betcard_log.json`, `live_alerts.json`, `lines.json` (open/last-before-kickoff lines kept a year). The Monday report summarizes all of them.
- Tonight's example (Bucs @ Cowboys): Under 48.5 and Bucs team total under 19.5, 0.5u each, both paper. Dallas has the NFL's worst defense; Marv's base projection counted it.

8. How an assistant works on the VM

The assistant cannot reach the VM directly. Both sides can reach GitHub, so jobs go through the **vm-link** branch: commit a one-line file `jobs/NAME.job`; the VM runner (every minute) executes it and commits `results/NAME.txt`. One job runs at a time. Allowed jobs: tests, update, status, logs UNIT [n], show PATH, telegram cards/NAME.txt [mirror], find-odds-bot, read-odds-bot /abs/folder, and marv COMMAND (run/props need `--dry-run`). A '# timeout SECONDS' line raises the default 45-minute limit. Secrets from `/opt/marv-bot/.env` are blanked from all output. The runner skips a job whose result file already exists, so a killed job is not retried automatically.

```
jobs/example.job: marv gaps
```

9. Current state and open items

Deployed vs not yet deployed

- **On the VM now (commit ba78662)**: market-anchored confidence, private-repo-ready updates (fetch over the deploy key), all tests passing (100).
- **Pushed to GitHub but NOT yet on the VM**: CFBD fixes (retry on timeouts/5xx, 3-hour response cache for the free-tier call cap, fall back to ESPN schedule if CFBD stays down, FCS games excluded from trend tags and the bet card). An update job (run7-update) is queued but blocked behind long backtests.
- **VM job queue**: run3-ncaab (college men's backtest) produced no result and was likely killed (possible out-of-memory after ~80 min); run4-ncaaw has been running since 11:07 ET; run5-wnba, run6-status, run7-update, run8-vm-link-log are queued behind it. Consider re-running ncaab in smaller season ranges, and put the update first.
- A scheduled check-in routine exists in the old assistant session; it will not carry over. Delete or ignore it.

Open work, in priority order

- Deploy the CFBD fix (this morning's college football run failed on an API timeout; the card still went out via the gap and trend paths).
- Make the repo private (owner agreed): the VM update path is ready (SSH deploy key at `/root/.ssh/marv_link`). After flipping GitHub visibility, run a vm-link update + status job to confirm. Docs and helper scripts that curl `raw.githubusercontent.com` will stop working and should be rewritten.

- March_edge (the owner's separate odds-alert bot, aj-bet-bot, /home/moonrider1968/aj-bet-bot): analyze and repair - overconfident/contradictory win chances, duplicate alerts, paper record vs ROI mismatch; add Marv /grade and a MARCH_EDGE label. Waiting on the owner choosing: 1 go, 2 private first, 3 VM session (deploy/claude-link.sh). Its Telegram token was exposed in chat earlier: it must be revoked in BotFather and replaced.
- Run the market-vs-model stack for NCAA/NCAAW/WNBA with historical lines, update RULES in marv/stats/registry.py and marv/backtest_results.json from state/reports.
- Backtest the injury-trend adjustment (marv/regime.py). Evaluate CLV and graded paper records (bet card tiers, trend tags, gaps) as data accumulates; none are proven.
- Live in-game props: snapshots are being recorded; backtest after 3-4 weeks before any live prop alerts. Verify ESPN live summary parsers (marv/ingame/plays.py) against a real response.
- EuroLeague Women (probe-ewl, marv/data/fiba.py), NCAA props, CFBD backtest (stats-backtest --sport cfb --seasons 2016-2025).

10. Rules and cautions

- Run `python -m unittest discover -s tests` before restarting services (104 tests currently).
- Keep PAPER_MODE on unless the owner explicitly turns it off.
- Never print or commit secrets from .env (Telegram tokens, Odds API key, CFBD key). The repo is currently public; remind the owner not to paste keys into chat. Chats are not end-to-end encrypted.
- Report results faithfully: if a test fails or a job dies, say so. Do not oversell accuracy numbers: they reflect heavy favorites, not profit.
- Commit messages and PR text must not name the AI model. Do not open a pull request unless the owner asks. Develop and push on claude/analysis-ak180w.
- Owner preferences: alerts only for games on the day (unless queried), only NEW edges with win probability, all sources labeled to avoid confusion between Marv and March_edge, college alerts included, mirror to @March_betbot on.

11. Key files

Path	Purpose
marv/main.py	CLI: run, live, edges, gaps, props, injuries, notify, stats-backtest, h2h-backtest, grade-prop, mirror-status, ...
marv/engine.py, marv/sims/	Projection -> Monte Carlo -> veto stack
marv/stats/ (live.py, registry.py, experts.py, features.py, backtest.py)	Stats experts, pick rules per sport, walk-forward backtests
marv/regime.py, marv/roster_notes.py, marv/data/injuries.py	Injury trends, roster notes, backup-QB flag
marv/betcard.py, marv/ou_tags.py, marv/sharpgap.py, marv/linemove.py	Daily bet card, O/U trend tags, Bovada vs Pinnacle gaps, line-move notes
marv/live_monitor.py, marv/live_props.py, marv/data/euroleague_live.py	In-game alerts and live prop recorder
marv/telegram.py, marv/alertday.py, marv/querybot.py, marv/bridge.py	Telegram sending/mirror, day-of rule, query bot, March_edge bridge
marv/prograde.py	Quick second opinion on any NFL player prop (/grade); grades stop at B

Path	Purpose
deploy/ (install.sh, vm_link.py, vm-link.sh, claude-link.sh, marv-health.sh)	Installer, VM link, health check
ANALYSIS.md, CLAUDE.md, README.md, BOOKS.md	Full analysis write-ups and handoff notes